

## часть 3

### Память

Опять будем смотреть рукомахательное количество информации но к другому вопросу - какая зависимость связи по времени, насколько чат помнит предыдущее (если процесс близок к марковскому то будет помнить фиксированное количество предыдущих и дальше падать очень быстро связь)

+проверим какой формы деревья ответов (можно же думать что это частично упорядоченное множество, можно исследовать характерные его величины - типа длина максимальной цепи, средняя ширина и вот такое)

Импорты

```
#копипаст из предыдущего
import os
while not os.path.isdir("chats"): os.chdir("..")
import numpy as np, pandas as pd, seaborn as sns

df = pd.read_csv("chats/physics/df.csv", parse_dates=["ts"])
df = df[~df["is_bot"]].reset_index(drop=True)
cats = sorted(df["mega"].unique())
K = len(cats)

df.iloc[0]

df["k"] = df["mega"].map({c: i for i, c in enumerate(cats)})
codes = df["k"].to_numpy()

codes[:20]

array([6, 6, 6, 6, 6, 6, 6, 6, 5, 5, 5, 5, 4, 4, 4, 4, 4, 3, 4, 4, 4])

{c: i for i, c in enumerate(cats)}

{'others': 0,
 'Баллы и апелляции': 1,
 'Логистика и быт': 2,
 'Мета': 3,
 'Оффтоп/болтовня': 4,
 'Поступление и подготовка': 5,
 'Разбор задач': 6}
```

### как далеко тянется память

идея: взять два сообщения на расстоянии  $d$  в ленте и измерить такую же взаимную информацию (KL дивергенцию между бернулевским процессом и наблюдаемой связью) - и это еще будет

функция от d

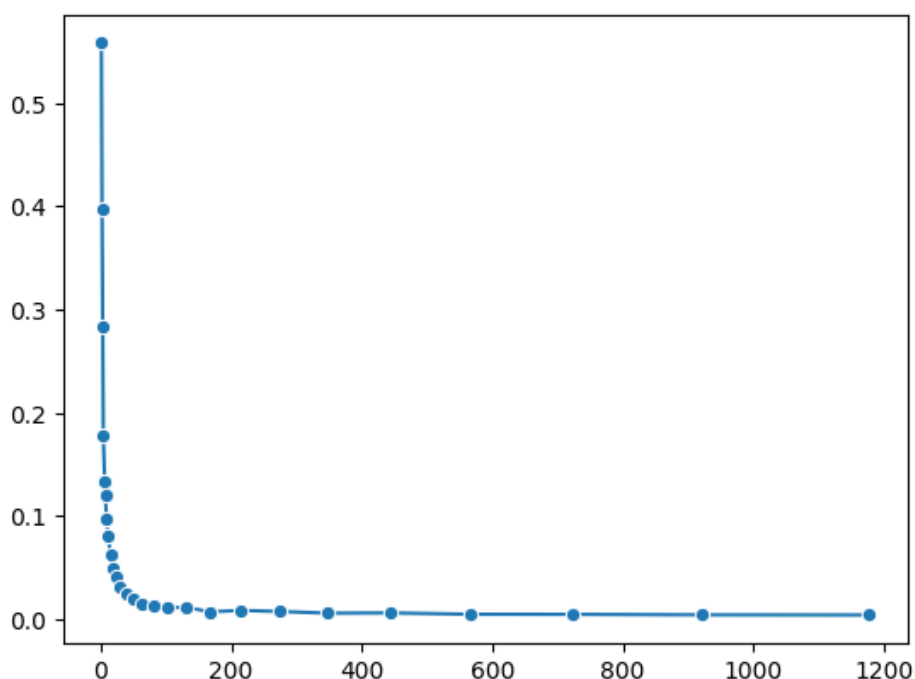
Можно посмотреть на асимптотику зависимости  $I(d)$

+как мы поняли что есть маленький шум связанный, с дисперсией самого процесса бернули, можно просто посчитать и вычесть его

```
#gemini {
def table(a, b):
    T = np.zeros((K, K)); np.add.at(T, (a, b), 1); return T
def MI(a, b):
    M = table(a, b); P = M / M.sum()
    Pi, Pj = P.sum(1, keepdims=True), P.sum(0, keepdims=True)
    m = P > 0
    return (P[m] * np.log2(P[m] / (Pi * Pj)[m])).sum() # KL(наблюдаемое ||
    ↪ бернулли p_i*p_j)

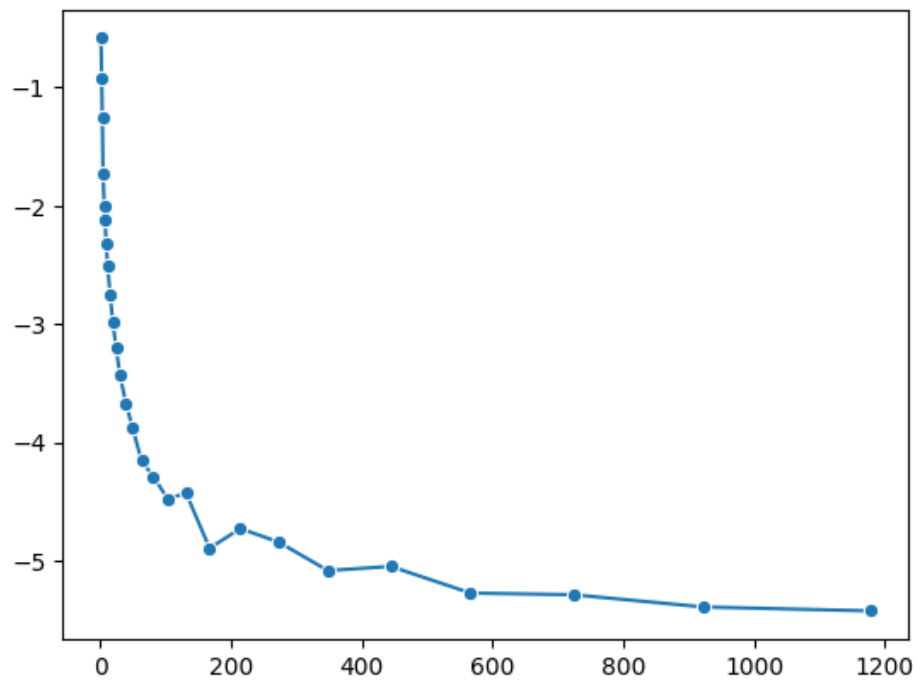
deltas = np.unique(np.round(np.logspace(0, np.log10(len(codes)//100),
    ↪ 30)).astype(int))

I = []
for d in deltas:
    I.append(MI(codes[:-d], codes[d:])) # связь тем на расстоянии d
I = np.array(I)
sns.lineplot(x=deltas, y=I, marker="o")
#}
```



ожидаемо падает очень быстро, сообщение теряется в чате и тд

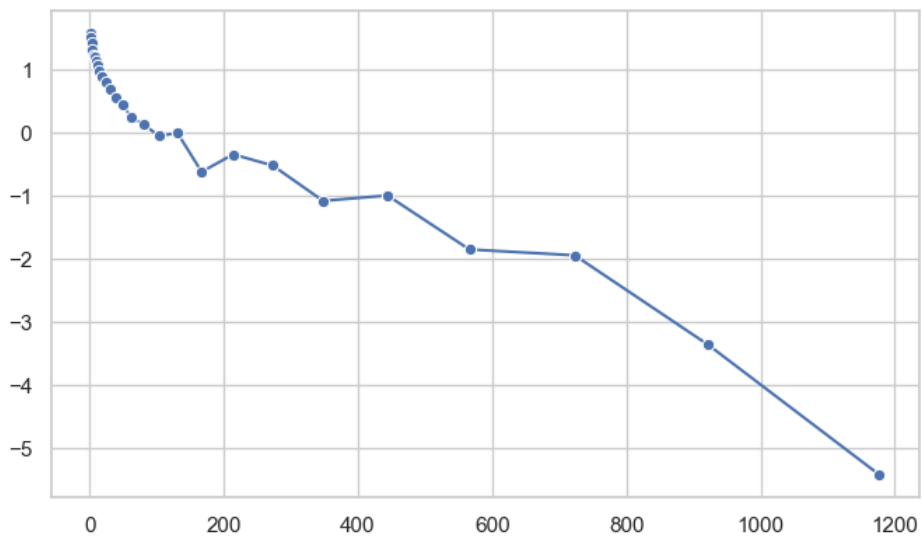
```
#логарифм
sns.lineplot(x=deltas, y=np.log(I), marker="o")
```



```
np.log(I)[-3:]
```

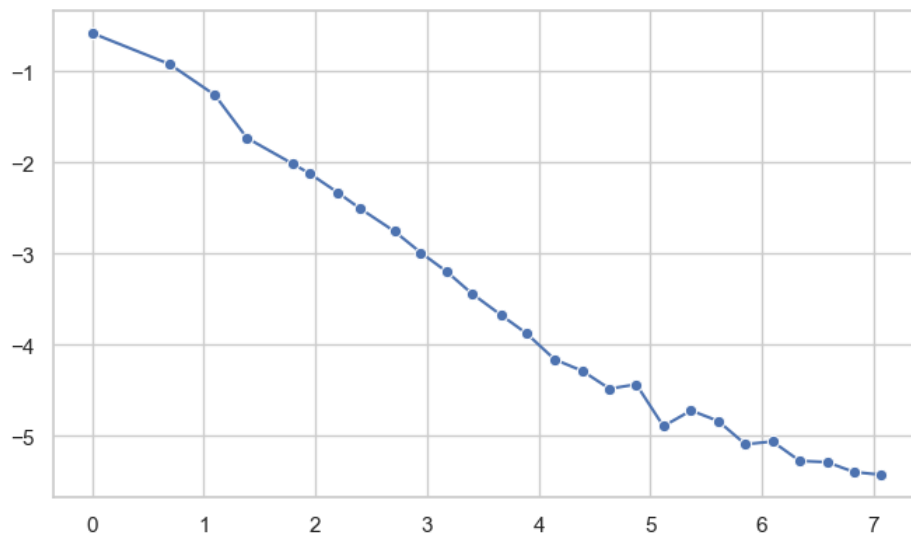
```
array([-5.28700326, -5.39543968, -5.42559768])
```

```
sns.lineplot(x=deltas, y=np.log(np.log(I) + 5.43), marker="o")
```



```
#похоже ли на прямую????
```

```
sns.lineplot(x=np.log(deltas), y=np.log(I), marker="o")
```



#вот это грамотно

#посчитаем коэффициент

```
k, b = np.polyfit(np.log(deltas), np.log(I), 1)
print(k, b)
```

```
-0.7302880080499937 -0.7639990795877946
```

1-0.73

0.27

### экспонента или степень?

то есть у нас  $\log(d) = A + k * \log(\delta)$

$d = C * \delta^k$  - степенная зависимость,  $k = -0.73$  - примерно обратная пропорциональность

#причем кстати прикольно что степеню одинаковую на всех масштабах  $d$ , то есть от  
 ↳ 10 до тысяч  
 #надо подумать связано ли это со свойствами KL дивергенции

### в близи на сколько шагов нужно помнить?

всм какое характерное число предыдущих сообщений дают существенную часть информации темы (типа интеграл от количества информации по  $d$ )

ИТОГО измерить сколько бит добавляет знание ЕЩЕ одного предыдущего сообщения, где добавка

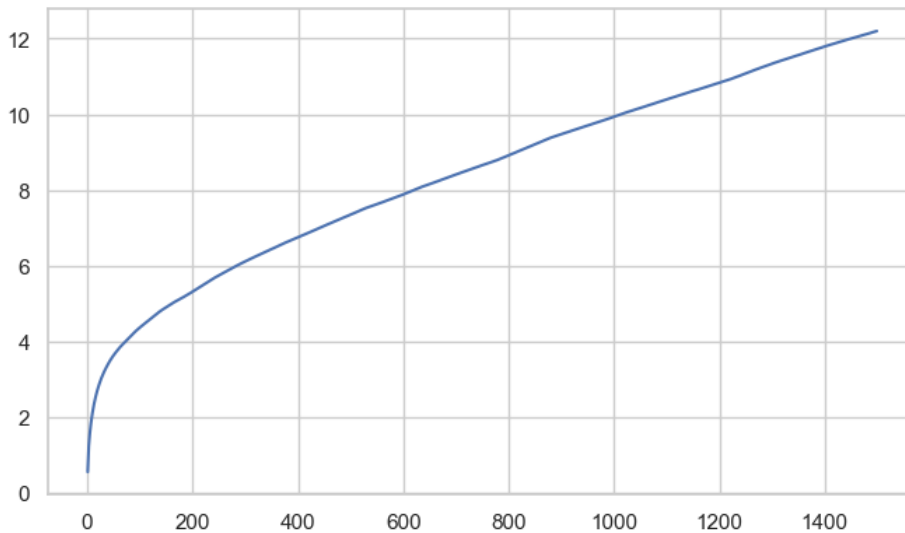
падает в ноль там можно остановиться (забить на хвосты - ИЛИ надо проверить что они не достаточно тяжелые!!!!!!)

оценка ручками:

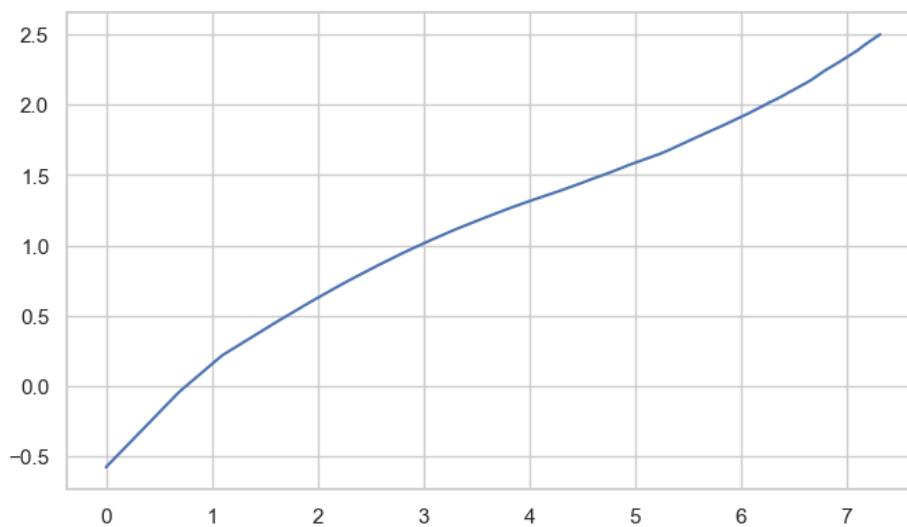
$I = C1 * d^k$ ,  $\text{Int}(I) = C2 * d^{(k+1)} \sim d^{0.27}$  - будет очень медленно ползти вверх неограниченно,

то есть ряд расходится и память бесконечная (прикольно)

```
dd = np.arange(1, 1500)
Id = np.array([MI(codes[:-d], codes[d:]) for d in dd])
s = np.cumsum(Id)
sns.lineplot(x=dd, y=s)
```



```
sns.lineplot(x=np.log(dd), y=np.log(S))
```



```
np.polyfit(np.log(dd[50:]), np.log(S[50:]), 1)[0] #k+1
```

```
np.float64(0.38216006990550627)
```

#кстати странно, у нас получилась небольшая ошибка в измерении k

### время возвращения темы

лемма Каца гласит что среднее время возвращения в состояние X - обратнопропорционально доле времени проводимому системой в этом состоянии

Можно взять одну тему и смотреть промежутки между ее появлениями

+надо смотреть именно на номера сообщений (то есть мы смотрим на строку где буква - тема)

- тк есть провалы по времени и тд и тп

+посмотреть не только среднее но глянуть и распределение этого времени ожидания

```
#посмотрим на топ 1
top = df["mega"].value_counts().index[0]
top

'Оффтоп/болтовня'

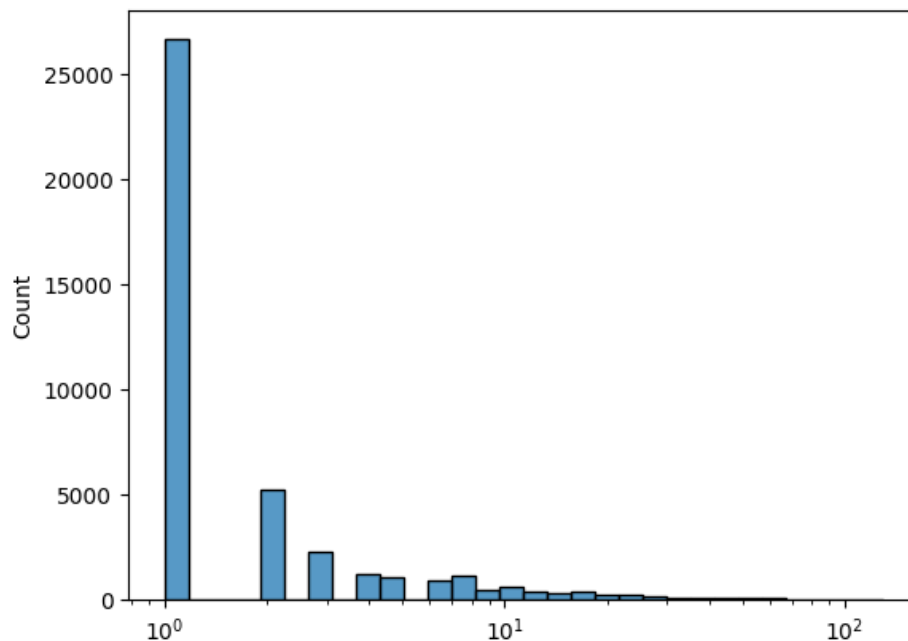
kt = cats.index(top)
kt

4

gaps = np.diff(np.where(codes == kt)[0])
gaps

array([1, 1, 1, ..., 1, 1, 1])

sns.histplot(gaps, log_scale=True, bins=30)
```



#длинный хвост, те тема может не появляться подолгу и резко очередями спамить

```
dictik = {i: c for i, c in enumerate(cats)}
dictik2 = {c: i for i, c in enumerate(cats)}
```

```
dictik
```

```
{0: 'others',
 1: 'Баллы и апелляции',
```

```

2: 'Логистика и быт',
3: 'Мета',
4: 'Офтоп/болтовня',
5: 'Поступление и подготовка',
6: 'Разбор задач'}

```

```
# по другим темам
```

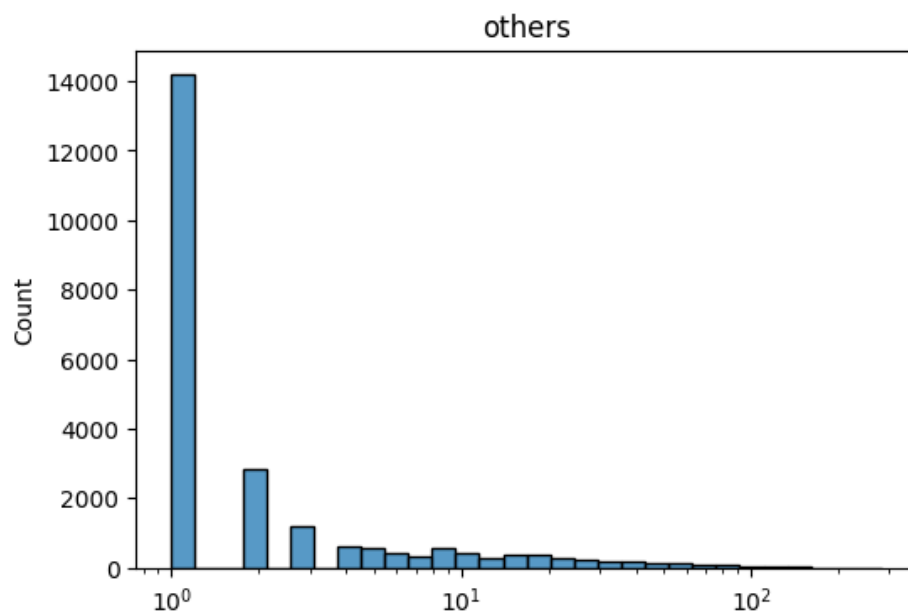
```
# gemini {
```

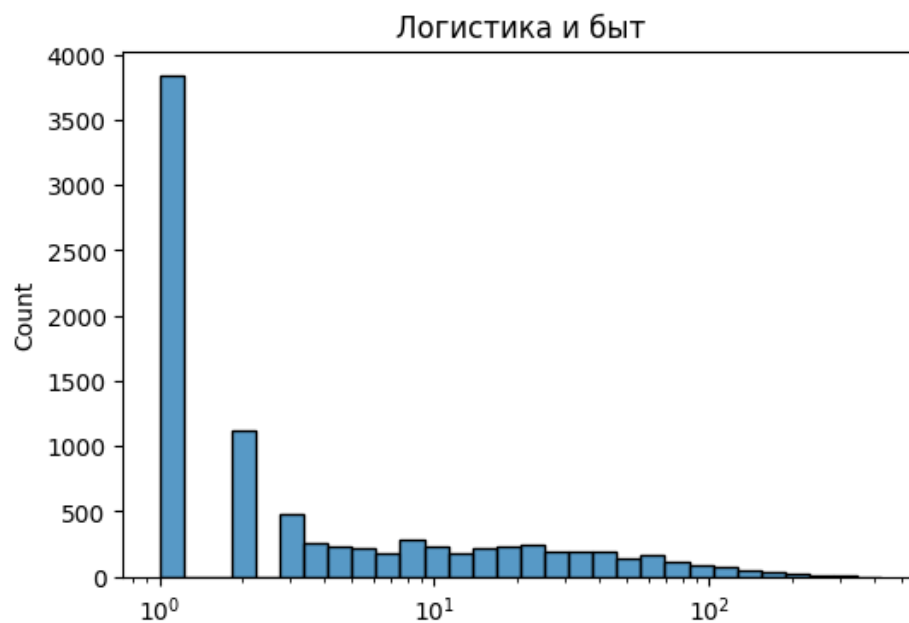
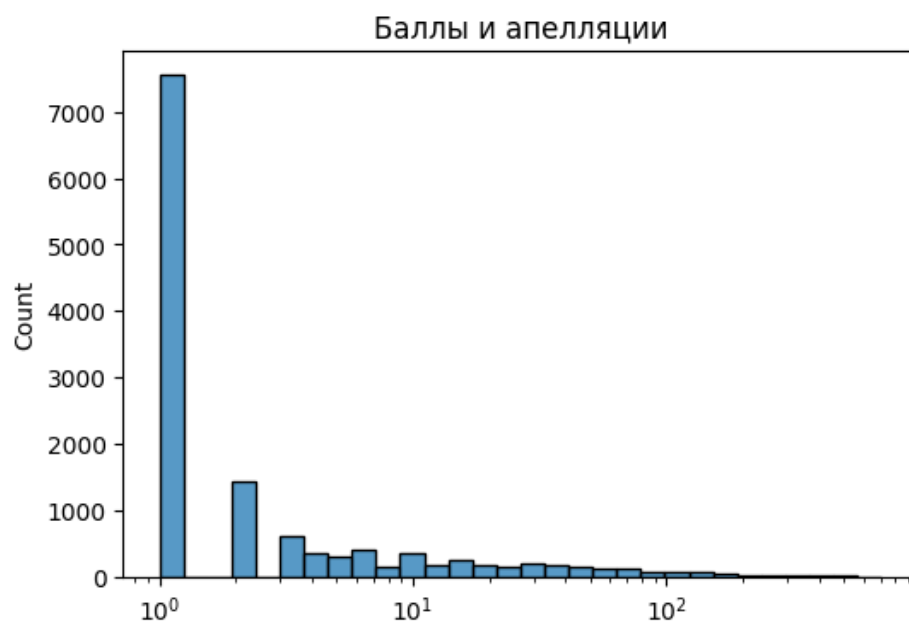
```
import matplotlib.pyplot as plt
```

```

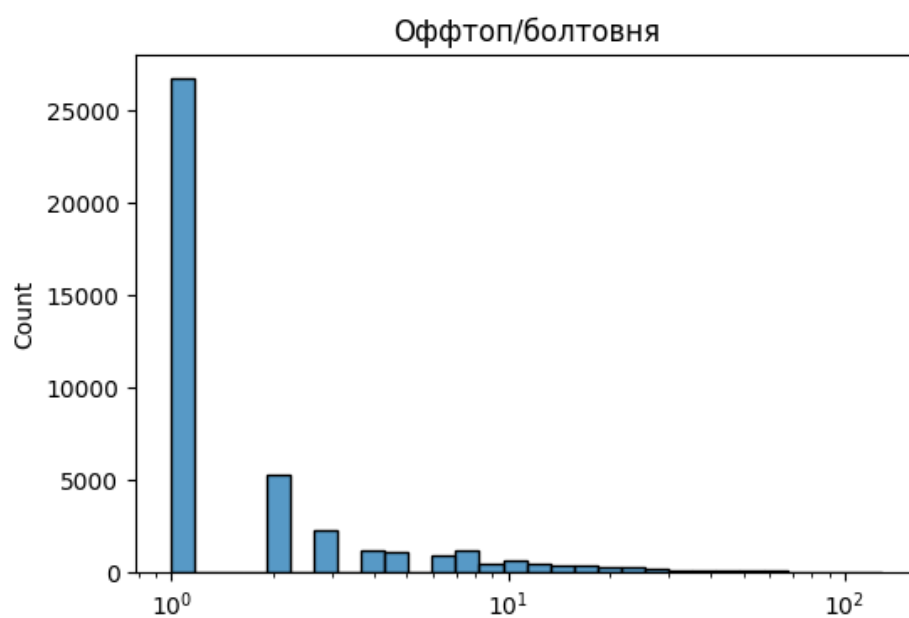
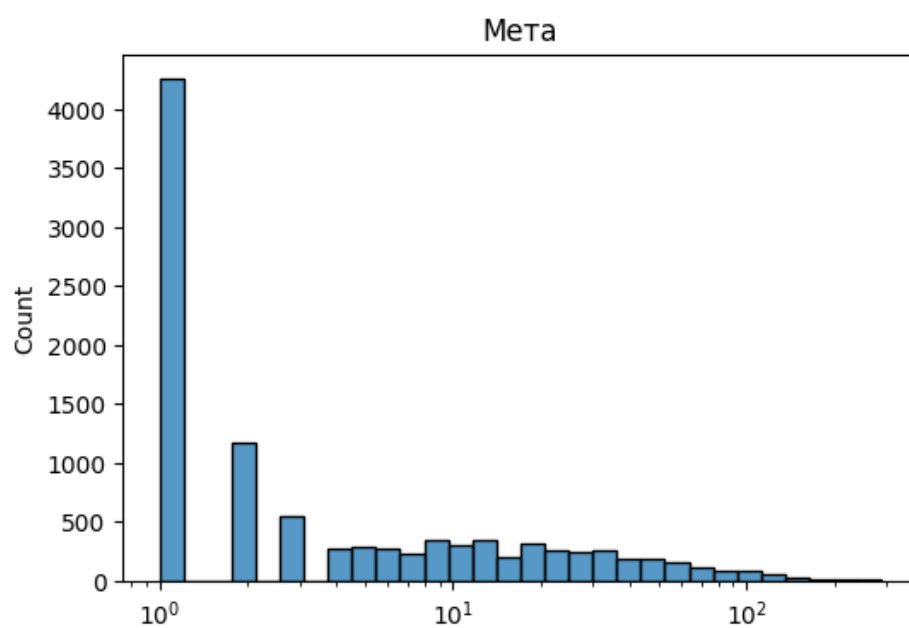
for kt in list(cats):
    plt.figure(figsize=(6, 4)) # Создаем новую фигуру, чтобы графики не слипались
    sns.histplot(np.diff(np.where(codes == dictik2[kt])[0]), log_scale=True,
        ↪ bins=30)
    plt.title(kt)           # Берем лейбл из словаря и ставим наверх
    plt.show()             # Отрисовываем текущий график перед переходом к
        ↪ следующему
#}

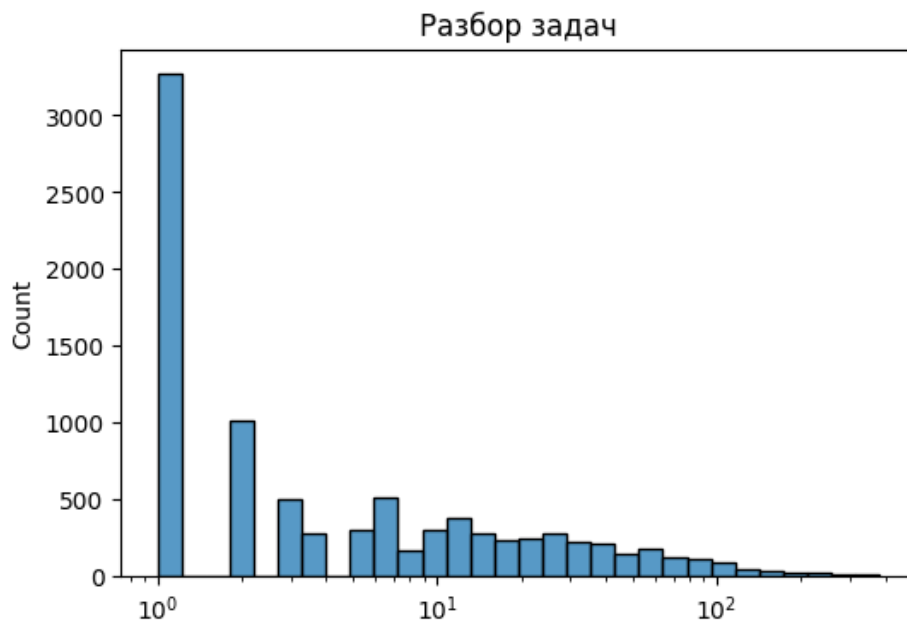
```











**примерно есть одно и то же** еще удобно ввести метрику  $B = (\sigma - \mu) / (\sigma + \mu)$  по промежуткам:

типа  $\sigma = \mu \rightarrow B=0$ ,  $B>0$  значит сообщения в основном идут пачками,  $B<0$  более равномерно, по расписанию и делим на сумму чтоб нормированно чтоб было от -1 до 1 в среднем

```
def B(seq, k):
    g = np.diff(np.where(seq == k)[0]).astype(float)
    return (g.std() - g.mean()) / (g.std() + g.mean())

perm = np.random.permutation(codes)
rows = []

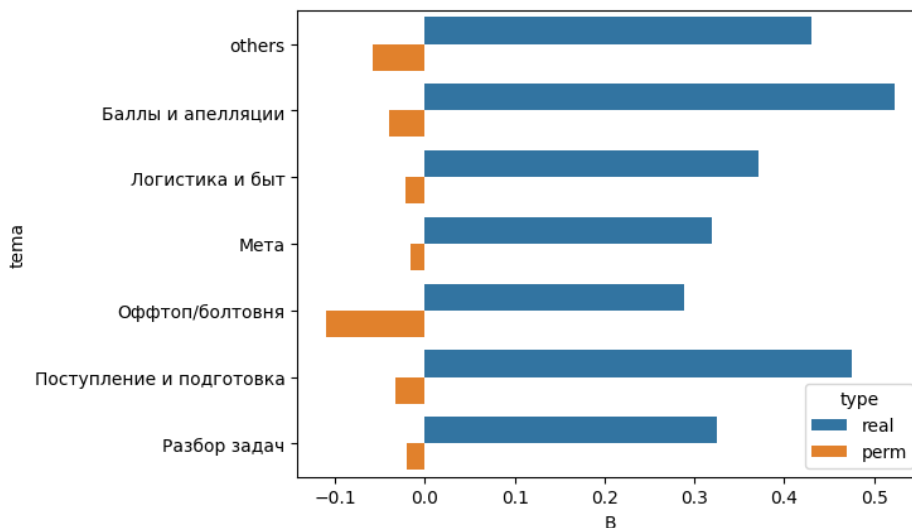
for k in range(K):
    rows.append({"tema": cats[k], "B": B(codes, k), "type": "real"})
    rows.append({"tema": cats[k], "B": B(perm, k), "type": "perm"})
```

```
rows = pd.DataFrame(rows)
```

```
rows
```

|    | tema                     | B         | type |
|----|--------------------------|-----------|------|
| 0  | others                   | 0.430766  | real |
| 1  | others                   | -0.057664 | perm |
| 2  | Баллы и апелляции        | 0.522471  | real |
| 3  | Баллы и апелляции        | -0.040042 | perm |
| 4  | Логистика и быт          | 0.371896  | real |
| 5  | Логистика и быт          | -0.021512 | perm |
| 6  | Мета                     | 0.320048  | real |
| 7  | Мета                     | -0.016114 | perm |
| 8  | Оффтоп/болтовня          | 0.288397  | real |
| 9  | Оффтоп/болтовня          | -0.110572 | perm |
| 10 | Поступление и подготовка | 0.475966  | real |
| 11 | Поступление и подготовка | -0.032864 | perm |
| 12 | Разбор задач             | 0.324558  | real |
| 13 | Разбор задач             | -0.019744 | perm |

```
sns.barplot(data=rows, x="B", y="tema", hue="type")
```



понятно что у перестановочного должно быть близко к пуассоновскому или другому хорошему где среднее близко с дисперсией

А настоящие - в основном идут пачками (в распределении очень большой столб на нуле и длинный хвост)

## форма деревьев обсуждений

ответы делают деревья (тк каждое сообщение отвечает максимум на одно!)

```
parent = dict(zip(df["unit_id"], df["reply_to_unit"]))
ids = set(df["unit_id"])

roots = {}
for u in df["unit_id"]:
```

```
x = u
while parent[x] in ids:
    x = parent[x]
roots[u] = x
```

roots

```
{0: 0,
1: 1,
2: 2,
3: 3,
4: 4,
5: 5,
6: 6,
7: 4,
8: 8,
9: 9,
10: 10,
11: 11,
12: 12,
13: 13,
14: 14,
15: 15,
16: 16,
17: 15,
18: 15,
19: 15,
20: 2,
21: 21,
22: 2,
23: 2,
24: 2,
25: 2,
26: 2,
27: 27,
28: 28,
29: 29,
30: 30,
31: 31,
32: 32,
33: 2,
34: 29,
35: 2,
36: 2,
37: 2,
38: 2,
39: 2,
40: 2,
41: 2,
42: 2,
43: 43,
44: 2,
45: 2,
46: 2,
47: 47,
48: 2,
49: 47,
50: 47,
51: 51,
52: 51,
53: 53,
54: 53,
```

55: 53,  
56: 56,  
57: 53,  
58: 58,  
59: 59,  
60: 60,  
61: 61,  
62: 62,  
63: 63,  
64: 63,  
65: 65,  
66: 66,  
67: 65,  
68: 65,  
69: 69,  
70: 70,  
71: 69,  
72: 70,  
73: 70,  
74: 74,  
75: 70,  
76: 70,  
77: 70,  
78: 70,  
79: 70,  
80: 80,  
81: 80,  
82: 80,  
83: 83,  
84: 84,  
85: 80,  
86: 80,  
87: 87,  
88: 83,  
89: 80,  
90: 87,  
91: 83,  
92: 70,  
93: 87,  
94: 80,  
95: 83,  
96: 87,  
97: 97,  
98: 98,  
99: 87,  
100: 87,  
101: 101,  
102: 87,  
103: 103,  
104: 87,  
105: 87,  
106: 87,  
107: 107,  
108: 108,  
109: 109,  
110: 110,  
111: 111,  
112: 112,  
113: 113,  
114: 111,  
115: 115,  
116: 113,

117: 117,  
118: 112,  
119: 119,  
120: 115,  
121: 121,  
122: 117,  
123: 117,  
124: 117,  
125: 125,  
126: 111,  
127: 127,  
128: 121,  
129: 129,  
130: 129,  
131: 131,  
132: 121,  
133: 131,  
134: 129,  
135: 131,  
136: 129,  
137: 129,  
138: 129,  
139: 129,  
140: 140,  
141: 141,  
142: 129,  
143: 143,  
144: 141,  
145: 141,  
146: 143,  
147: 141,  
148: 148,  
149: 148,  
150: 150,  
151: 70,  
152: 150,  
153: 150,  
154: 148,  
155: 148,  
156: 148,  
157: 148,  
158: 70,  
159: 70,  
160: 70,  
161: 161,  
162: 148,  
163: 148,  
164: 148,  
165: 70,  
166: 150,  
167: 150,  
168: 150,  
169: 169,  
170: 169,  
171: 169,  
172: 172,  
173: 173,  
174: 172,  
175: 169,  
176: 169,  
177: 169,  
178: 173,

179: 169,  
180: 150,  
181: 173,  
182: 173,  
183: 169,  
184: 169,  
185: 173,  
186: 169,  
187: 173,  
188: 169,  
189: 150,  
190: 169,  
191: 169,  
192: 169,  
193: 150,  
194: 150,  
195: 169,  
196: 196,  
197: 197,  
198: 150,  
199: 199,  
200: 150,  
201: 201,  
202: 199,  
203: 169,  
204: 201,  
205: 201,  
206: 169,  
207: 207,  
208: 199,  
209: 209,  
210: 210,  
211: 210,  
212: 209,  
213: 210,  
214: 209,  
215: 210,  
216: 210,  
217: 210,  
218: 209,  
219: 210,  
220: 210,  
221: 221,  
222: 221,  
223: 210,  
224: 210,  
225: 225,  
226: 226,  
227: 225,  
228: 210,  
229: 229,  
230: 226,  
231: 231,  
232: 225,  
233: 226,  
234: 226,  
235: 235,  
236: 226,  
237: 226,  
238: 226,  
239: 239,  
240: 240,

241: 240,  
242: 239,  
243: 240,  
244: 240,  
245: 245,  
246: 240,  
247: 245,  
248: 248,  
249: 249,  
250: 240,  
251: 251,  
252: 252,  
253: 253,  
254: 252,  
255: 253,  
256: 256,  
257: 240,  
258: 253,  
259: 256,  
260: 210,  
261: 261,  
262: 262,  
263: 263,  
264: 262,  
265: 261,  
266: 263,  
267: 263,  
268: 262,  
269: 262,  
270: 270,  
271: 262,  
272: 270,  
273: 262,  
274: 262,  
275: 262,  
276: 270,  
277: 262,  
278: 262,  
279: 270,  
280: 262,  
281: 270,  
282: 270,  
283: 270,  
284: 270,  
285: 270,  
286: 270,  
287: 270,  
288: 270,  
289: 289,  
290: 290,  
291: 291,  
292: 290,  
293: 291,  
294: 290,  
295: 295,  
296: 291,  
298: 291,  
299: 291,  
301: 291,  
302: 302,  
303: 303,  
304: 302,



305: 305,  
306: 305,  
307: 307,  
308: 307,  
309: 309,  
310: 310,  
311: 307,  
312: 312,  
313: 313,  
314: 307,  
315: 315,  
316: 316,  
317: 307,  
318: 315,  
319: 307,  
320: 320,  
321: 321,  
322: 307,  
323: 315,  
324: 307,  
325: 325,  
326: 315,  
327: 327,  
328: 315,  
329: 329,  
330: 330,  
331: 321,  
332: 315,  
333: 315,  
334: 321,  
335: 335,  
336: 321,  
337: 330,  
338: 338,  
339: 330,  
340: 321,  
341: 321,  
342: 321,  
343: 321,  
344: 344,  
345: 345,  
346: 345,  
347: 345,  
348: 348,  
349: 349,  
350: 350,  
351: 350,  
352: 350,  
353: 350,  
354: 350,  
355: 350,  
356: 350,  
357: 345,  
358: 350,  
359: 345,  
360: 345,  
361: 350,  
362: 350,  
363: 350,  
364: 350,  
365: 350,  
366: 350,

```

367: 350,
368: 350,
369: 369,
370: 350,
371: 350,
372: 372,
373: 372,
374: 350,
375: 372,
376: 372,
377: 372,
378: 350,
379: 350,
380: 380,
381: 372,
382: 350,
383: 383,
384: 384,
385: 372,
386: 384,
387: 384,
38

```

```

sizes = pd.Series(roots).value_counts()
sizes

```

```

100206      51
86548       51
23534       50
372         49
6950        46
..
53954        1
53957        1
53958        1
53959        1
120124       1
Name: count, Length: 52828, dtype: int64

```

```

vc = sizes.value_counts()
vc

```

```

count
1      32228
2       8899
3       3951
4       2353
5       1453
6        971
7        659
8        492
9        381
10       283
11       197
12       188
13       146
14       123
15        81
17        58
16        57
18        46

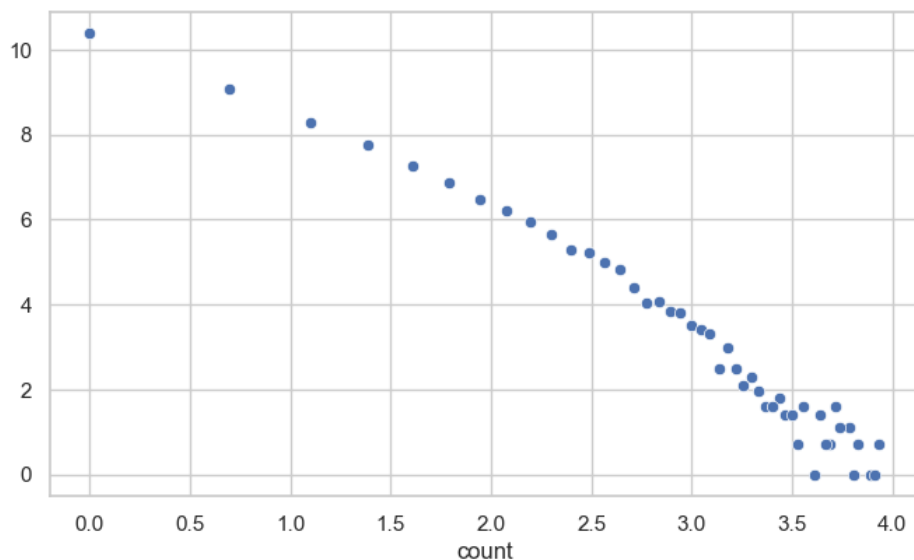
```

```

19      45
20      33
21      30
22      27
24      20
25      12
23      12
27      10
26       8
28       7
31       6
35       5
41       5
29       5
30       5
32       4
33       4
38       4
44       3
42       3
46       2
51       2
40       2
39       2
34       2
37       1
45       1
49       1
50       1
Name: count, dtype: int64

```

```
sns.scatterplot(x=np.log(vc.index), y=np.log(vc.values))
```



```

#распределение количества тредов определенной длины от их длины (в лог шкале)
#то есть сильно большая (на 32к!) есть но остальные на порядки меньше и в среднем
↳ все маленькие

```

давайте померим насколько параллельны эти истории - если взять 2 случайных сообщения (либо во времени чтоб они близко были) - какая вероятность что они будут сравнимы?

ну мы будем брать 2 случайных сообщения в некотором окне длины  $t$  и можно как раз померить зависимость этой вероятности от  $t$

```
N = len(df)
N
```

```
117775
```

```
pos = dict(zip(df["unit_id"], range(N)))
pos
```

```
{0: 0,
 1: 1,
 2: 2,
 3: 3,
 4: 4,
 5: 5,
 6: 6,
 7: 7,
 8: 8,
 9: 9,
10: 10,
11: 11,
12: 12,
13: 13,
14: 14,
15: 15,
16: 16,
17: 17,
18: 18,
19: 19,
20: 20,
21: 21,
22: 22,
23: 23,
24: 24,
25: 25,
26: 26,
27: 27,
28: 28,
29: 29,
30: 30,
31: 31,
32: 32,
33: 33,
34: 34,
35: 35,
36: 36,
37: 37,
38: 38,
39: 39,
40: 40,
41: 41,
42: 42,
43: 43,
44: 44,
45: 45,
46: 46,
47: 47,
48: 48,
```

49: 49,  
50: 50,  
51: 51,  
52: 52,  
53: 53,  
54: 54,  
55: 55,  
56: 56,  
57: 57,  
58: 58,  
59: 59,  
60: 60,  
61: 61,  
62: 62,  
63: 63,  
64: 64,  
65: 65,  
66: 66,  
67: 67,  
68: 68,  
69: 69,  
70: 70,  
71: 71,  
72: 72,  
73: 73,  
74: 74,  
75: 75,  
76: 76,  
77: 77,  
78: 78,  
79: 79,  
80: 80,  
81: 81,  
82: 82,  
83: 83,  
84: 84,  
85: 85,  
86: 86,  
87: 87,  
88: 88,  
89: 89,  
90: 90,  
91: 91,  
92: 92,  
93: 93,  
94: 94,  
95: 95,  
96: 96,  
97: 97,  
98: 98,  
99: 99,  
100: 100,  
101: 101,  
102: 102,  
103: 103,  
104: 104,  
105: 105,  
106: 106,  
107: 107,  
108: 108,  
109: 109,  
110: 110,

111: 111,  
112: 112,  
113: 113,  
114: 114,  
115: 115,  
116: 116,  
117: 117,  
118: 118,  
119: 119,  
120: 120,  
121: 121,  
122: 122,  
123: 123,  
124: 124,  
125: 125,  
126: 126,  
127: 127,  
128: 128,  
129: 129,  
130: 130,  
131: 131,  
132: 132,  
133: 133,  
134: 134,  
135: 135,  
136: 136,  
137: 137,  
138: 138,  
139: 139,  
140: 140,  
141: 141,  
142: 142,  
143: 143,  
144: 144,  
145: 145,  
146: 146,  
147: 147,  
148: 148,  
149: 149,  
150: 150,  
151: 151,  
152: 152,  
153: 153,  
154: 154,  
155: 155,  
156: 156,  
157: 157,  
158: 158,  
159: 159,  
160: 160,  
161: 161,  
162: 162,  
163: 163,  
164: 164,  
165: 165,  
166: 166,  
167: 167,  
168: 168,  
169: 169,  
170: 170,  
171: 171,  
172: 172,

173: 173,  
174: 174,  
175: 175,  
176: 176,  
177: 177,  
178: 178,  
179: 179,  
180: 180,  
181: 181,  
182: 182,  
183: 183,  
184: 184,  
185: 185,  
186: 186,  
187: 187,  
188: 188,  
189: 189,  
190: 190,  
191: 191,  
192: 192,  
193: 193,  
194: 194,  
195: 195,  
196: 196,  
197: 197,  
198: 198,  
199: 199,  
200: 200,  
201: 201,  
202: 202,  
203: 203,  
204: 204,  
205: 205,  
206: 206,  
207: 207,  
208: 208,  
209: 209,  
210: 210,  
211: 211,  
212: 212,  
213: 213,  
214: 214,  
215: 215,  
216: 216,  
217: 217,  
218: 218,  
219: 219,  
220: 220,  
221: 221,  
222: 222,  
223: 223,  
224: 224,  
225: 225,  
226: 226,  
227: 227,  
228: 228,  
229: 229,  
230: 230,  
231: 231,  
232: 232,  
233: 233,  
234: 234,

235: 235,  
236: 236,  
237: 237,  
238: 238,  
239: 239,  
240: 240,  
241: 241,  
242: 242,  
243: 243,  
244: 244,  
245: 245,  
246: 246,  
247: 247,  
248: 248,  
249: 249,  
250: 250,  
251: 251,  
252: 252,  
253: 253,  
254: 254,  
255: 255,  
256: 256,  
257: 257,  
258: 258,  
259: 259,  
260: 260,  
261: 261,  
262: 262,  
263: 263,  
264: 264,  
265: 265,  
266: 266,  
267: 267,  
268: 268,  
269: 269,  
270: 270,  
271: 271,  
272: 272,  
273: 273,  
274: 274,  
275: 275,  
276: 276,  
277: 277,  
278: 278,  
279: 279,  
280: 280,  
281: 281,  
282: 282,  
283: 283,  
284: 284,  
285: 285,  
286: 286,  
287: 287,  
288: 288,  
289: 289,  
290: 290,  
291: 291,  
292: 292,  
293: 293,  
294: 294,  
295: 295,  
296: 296,



298: 297,  
299: 298,  
301: 299,  
302: 300,  
303: 301,  
304: 302,  
305: 303,  
306: 304,  
307: 305,  
308: 306,  
309: 307,  
310: 308,  
311: 309,  
312: 310,  
313: 311,  
314: 312,  
315: 313,  
316: 314,  
317: 315,  
318: 316,  
319: 317,  
320: 318,  
321: 319,  
322: 320,  
323: 321,  
324: 322,  
325: 323,  
326: 324,  
327: 325,  
328: 326,  
329: 327,  
330: 328,  
331: 329,  
332: 330,  
333: 331,  
334: 332,  
335: 333,  
336: 334,  
337: 335,  
338: 336,  
339: 337,  
340: 338,  
341: 339,  
342: 340,  
343: 341,  
344: 342,  
345: 343,  
346: 344,  
347: 345,  
348: 346,  
349: 347,  
350: 348,  
351: 349,  
352: 350,  
353: 351,  
354: 352,  
355: 353,  
356: 354,  
357: 355,  
358: 356,  
359: 357,  
360: 358,

```

361: 359,
362: 360,
363: 361,
364: 362,
365: 363,
366: 364,
367: 365,
368: 366,
369: 367,
370: 368,
371: 369,
372: 370,
373: 371,
374: 372,
375: 373,
376: 374,
377: 375,
378: 376,
379: 377,
380: 378,
381: 379,
382: 380,
383: 381,
384: 382,
385: 3

```

```

gaps = []
for u in df["unit_id"]:
    x = u
    while parent[x] in ids:
        x = parent[x]
        gaps.append(pos[u] - pos[x])
gaps = np.array(gaps)
gaps

```

```
array([3, 2, 1, ..., 5, 6, 7])
```

```

ts = np.unique(np.round(np.logspace(0, np.log10(N//5), 25)).astype(int))
frac = []
for t in ts:
    comp = (gaps <= t).sum()           # сравнимых пар в окне t
    total = N * t - t*(t+1)//2        # всего пар (i<j, j-i<=t)
    frac.append(comp / total)

```

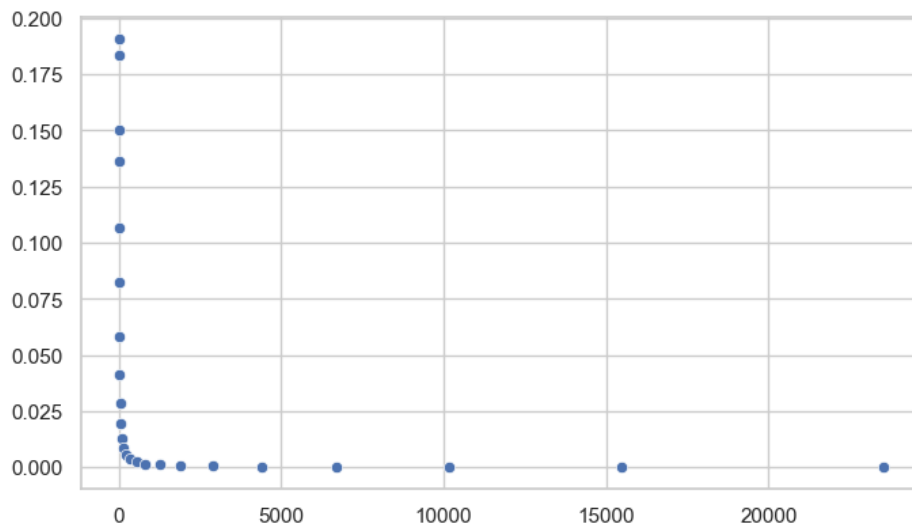
```
frac[:3]
```

```

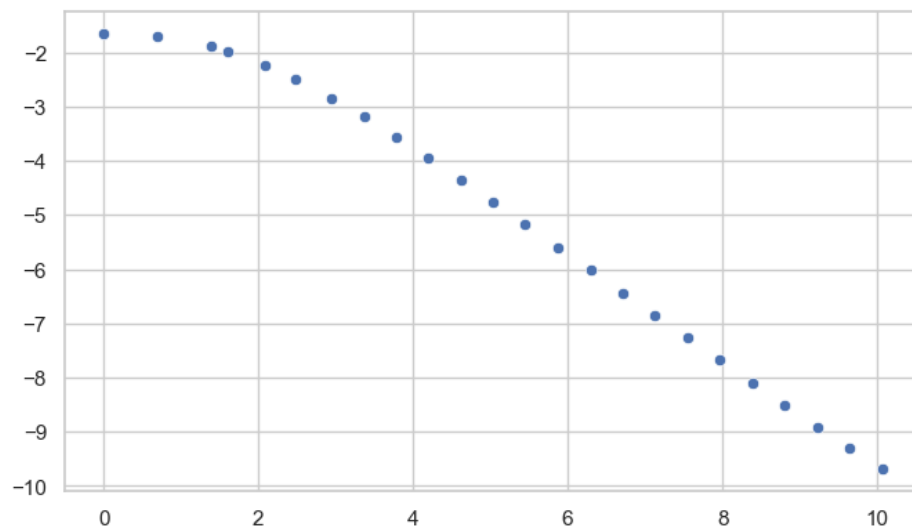
[np.float64(0.19071272097406897),
 np.float64(0.18346656930463984),
 np.float64(0.1501093209365514)]

```

```
sns.scatterplot(x=ts, y=frac)
```



```
sns.scatterplot(x=np.log(ts), y=np.log(frac))
```



```
k, b = np.polyfit(np.log(ts), np.log(frac), 1)
print(k, b)
```

```
-0.8775614672207555 -0.5914338579446607
```

**то есть  $\text{frac} \sim t^{-0.6}$**

### итоги по идеям

1. память степенная ( $I(d) \sim d^{-0.7}$ ) - при практически всех  $d$
2. нет характерного числа сообщений, которые сообщают информацию (ряд расходится)
3. все темы возвращаются пачками ( $B > 0$ ) - те же волны, что видели в части 1 (реальная дисперсия больше бернулевской теоретической)
4. много глубоких цепей ответов, но в основном короткие (количество также распределено по степенному закону от длины)
5. деревья ответов широкие ( $r \sim 1e-5$  на всем чате), и по степенному закону растет их ширина в зависимости от длины контекста